Package 'okwaayeli'

August 9, 2026

Type Package

Title Agricultural Productivity in Ghana

Version 0.0.0.9000

Author Francis Tsiboe [aut, cre] (<https://orcid.org/0000-0001-5984-1072>)

Maintainer Francis Tsiboe <ftsiboe@hotmail.com>

Contributor -

Reviewer -

Creator Francis Tsiboe

Description Provides tools and datasets for investigating the drivers of agricultural production shortfalls in Ghana.
It compiles research studies that examine farmer-specific and institutional factors to assess whether inefficiencies arise from technical inefficiency, technology gaps, or both. The package offers empirical evidence to inform policy discussions and interventions aimed at improving agricultural productivity, particularly in contexts with limited access to modern technologies.

License GPL-3 + file LICENSE

URL https://github.com/ftsiboe/okwaayeli

BugReports https://github.com/ftsiboe/okwaayeli/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

VignetteBuilder knitr

Depends R (>= 4.1.0)

Imports MatchIt, data.table, haven, stats, utils, sfaR, micEcon, piggyback, ggplot2, gridExtra, gtable, cowplot, magrittr, dplyr, stringr

Remotes github::dylan-turner25/rfcip

Suggests knitr, flextable, crayon, tidyr, withr, rmarkdown, lavaan, quadprog, emdbook, cobalt, cli, doBy, MASS, Matrix, corpcor, testthat (>= 3.0.0), car, frontier, rgenoud, purrr, randomForest, CBPS, dbarts, optmatch, Matching, classInt, fixest, marginaleffects, readxl, officer

LazyData true

Cite-us If you find it useful, please consider staring the repository and citing the following studies
- Tsiboe F. (2021). Chronic sources of low cocoa production in Ghana, new insights from meta-analysis of old survey data.
Agricultural and Resource Economics Review.50(2) 226-251.
- Tsiboe, F., Egyir, I. S., & Anaman, G. (2021). Effect of fertilizer subsidy on household level cereal production in Ghana.
Scientific African, 13, e00916.
- Tsiboe F. (2022). Nationally Representative Farm/Household Level Dataset on Crop Production in Ghana from 1987-2017.

Config/roxygen2/version 8.0.0

RoxygenNote 7.3.3

Contents

`compute_jenks_binned_shares`
Compute Jenks-Binned Shares by Aggregation Groups

Description

This function bins a numeric variable using Jenks (Fisher) natural breaks and computes the relative share of *counts* and/or *weights* within each aggregation group across the binned ranges.

It is useful for summarizing the distribution of a variable (e.g., dealer density, yield, acreage, distances) across spatial or categorical groups.

Usage

```
compute_jenks_binned_shares(
  dt,
  output_variable,
  aggregation_points,
  jenks = NULL,
  jenks_number = NULL,
  weight_variable = NULL
)
```

Arguments

`dt`	A `data.table` containing the data.
`output_variable`	Character name of the numeric column to bin.
`aggregation_points`	Character vector of grouping variables (e.g., `"region"`, `"district"`, `"grid_id"`).
`jenks`	Optional numeric vector of breakpoints. If `NULL`, Jenks breaks are computed automatically.
`jenks_number`	Integer number of Jenks classes to compute when `jenks` is `NULL`.
`weight_variable`	Optional character name of a weighting column. If `NULL`, a weight of `1` is assumed for each row.

Details

Internally the function:

1. Computes Jenks natural breaks (if not provided)
2. Creates `count` and `weight` totals within each bin by group
3. Merges with total counts/weights per group
4. Computes shares

Value

A `data.table` with:

- grouping variables
- `binned_range_name` - character label of the Jenks bin
- `binned_range_level` - numeric factor level of the bin
- `estimate_count` - share of counts within the group
- `estimate_weight` - share of weights within the group

See Also

Other helpers: `get_crop_area_list()`, `harmonized_data_prep()`

`covariate_balance` *Compute Covariate Balance Summaries Across Matching Specs*

Description

Compute Covariate Balance Summaries Across Matching Specs

Usage

```
covariate_balance(match_specifications, matching_output_directory)
```

Arguments

`match_specifications`	data.frame/data.table with at least columns: `boot`, `ARRAY`, and the spec fields stored in RDS (e.g., method, distance, link).
`matching_output_directory`	Directory containing RDS files named as "0001.rds", "0002.rds", etc.

Details

Reads each matching result RDS (expected to contain `m.out` and `match_specifications`), extracts balance via `cobalt::bal.tab`, reshapes, computes a composite balance $rate = mean((Diff - 0)^2, (KS - 0)^2, (V_Rratio - 1)^2)$, and averages by `ARRAY`, `method`, `distance`, `link`, `sample`.

Value

A list with:

`rate`	data.frame of mean balance metrics by spec (Adj sample only) with a composite `rate`.
`bal_tab`	long-format balance table per covariate/stat/sample/spec.

See Also

Other matching: `draw_matched_samples()`, `match_sample_specifications()`, `treatment_effect_calculation()`, `treatment_effect_summary()`, `write_match_formulas()`

descriptive_expand_category

Expand a Categorical Column into Indicator Dummies

Description

Reproduces Stata's `tab <var>, gen(<var>_)`: one 0/1 column per level, named `<var>_1`, `<var>_2`, ... in level order.

Usage

```
descriptive_expand_category(data, category_var)
```

Arguments

`data`	`data.frame`.
`category_var`	Character. Categorical column, e.g. `"LndOwn"`.

Value

`data` with the dummy columns appended. The new names are returned in `attr(x, "indicators")`.

See Also

Other descriptive exhibits: `descriptive_group_summary()`, `descriptive_indicator_shares()`, `descriptive_prepare()`, `descriptive_season_year()`, `descriptive_specifications()`, `descriptive_trend_m` `descriptive_workhorse()`, `draw_descriptive_summary()`

descriptive_group_summary

Weighted Summary Statistics by Group

Description

Engine A's aggregation step: mean, standard error of the mean, min, max, standard deviation and n, computed overall and by treatment group, and again within each survey wave. Replaces Stata's `tabstat ..., stat(mean sem min max sd n) by()`.

Usage

```
descriptive_group_summary(
  data,
  outcome,
  treatment = NULL,
  wave_var = "Surveyx",
  weights = NULL
)
```

Arguments

data	data.frame, already subset to one crop.
outcome	Character. Outcome column.
treatment	Character or NULL. Binary treatment column. NULL emits pooled rows only (Engine B).
wave_var	Character. Default "Surveyx".
weights	Character or NULL. Column of sampling weights.

Details

With weights = NULL (default) this reproduces the Stata path exactly: no 100_*.do in the repository applies sampling weights, despite WeightHH being available. Supplying weights = "WeightHH" yields population-representative means; see the *Divergences* section of studies/README_descriptive_exhibits_plar

Value

A data.frame with wave, group, statistic = "mean", estimate, se, min, max, sd, n.

Rows with a missing treatment are dropped

When treatment is supplied, **every** row it emits – including the pooled row – is restricted to observations with a non-missing treatment. This matches tabstat <var>, by(<treatment>), which excludes observations whose by-variable is missing, and it keeps the pooled row on the same sample as the group rows.

It matters. In resource_extraction the commercial/informal mining split exists only in GLSS6/GLSS7, so mining_comm and mining_gala are missing for all 10,416 GLSS4/GLSS5 observations. Pooling over every row regardless gives n = 26,811 against Stata's 16,395, and a mean drawn from a different population than the groups it sits beside.

See Also

Other descriptive exhibits: descriptive_expand_category(), descriptive_indicator_shares(), descriptive_prepare(), descriptive_season_year(), descriptive_specifications(), descriptive_trend_m descriptive_workhorse(), draw_descriptive_summary()

---

descriptive_indicator_shares

Indicator Shares by Wave (Engine B)

Description

Returns the share of each binary indicator overall, optionally by wave, plus a trend. Produces the rows behind Table 2 and its crop-disaggregated appendix analogues (land's Tables S1–S4, resource_extraction's extraction table).

Usage

```
descriptive_indicator_shares(
  data,
  indicators,
  trend = c("continuous", "wave_diff", "none"),
  waves = NULL,
  per_wave = TRUE,
  wave_var = "Surveyx",
  weights = NULL
)
```

Arguments

data	data.frame from descriptive_prepare(), subset to one crop.
indicators	Character vector of binary columns. Use descriptive_expand_category() first for a categorical source.
trend	"continuous", "wave_diff" or "none". See *Two trend flavors*.
waves	Character vector of length 2, c(from, to), required when trend = "wave_diff".
per_wave	Logical. Emit per-wave shares as well as the pooled share. TRUE for land, FALSE for resource_extraction (whose do-file collects only StatTotal).
wave_var	Character. Default "Surveyx".
weights	Character or NULL.

Value

A data.frame with outcome, wave, group = NA, statistic, estimate, se, min, max, sd, n. The pooled share is emitted with wave = "pooled" (the sheets call it GLSS0); the trend with wave = "trend".

Two trend flavors

The studies do **not** use the same estimator here, and the difference is substantive rather than cosmetic:

trend = "continuous" **(resource_extraction)** logit <ind> Trend then margins, eydx(Trend) predict(pr) and nlcom (_b[Trend]*100). Emits a **semi-elasticity**: percent change in the probability per year. Only the pooled share (GLSS0) accompanies it.

trend = "wave_diff" **(land_tenure)** logit <ind> i.Survey then margins Survey and nlcom ((_b[6bn.Survey]-_ Emits a **percentage-point** difference between two waves, and per-wave shares accompany it. Note the sign: it is **earlier minus later**, so an indicator that grew reports a negative value. Land's Table 2 shows Not owned = -9.743 while the share rose from 0.290 to 0.388. That is the convention, not an error.

Mixing them up yields plausible numbers on the wrong scale – percent per year versus percentage points – so trend has no default.

See Also

Other descriptive exhibits: descriptive_expand_category(), descriptive_group_summary(), descriptive_prepare(), descriptive_season_year(), descriptive_specifications(), descriptive_trend_m descriptive_workhorse(), draw_descriptive_summary()

`descriptive_prepare` *Add the Trend and Cluster Columns Used by the Descriptive Engines*

Description

Reproduces the two derived columns every `100_*.do` builds before estimating: `Trend = Season - min(Season)` and a cluster id grouping `Survey Ecozon EaId HhId`.

Usage

```
descriptive_prepare(
  data,
  season_var = "Season",
  cluster_vars = c("Survey", "Ecozon", "EaId", "HhId")
)
```

Arguments

`data`	`data.frame` subset (normally one crop).
`season_var`	Character. Default `"Season"`. Numeric years, or a factor/character whose labels begin with a four-digit year.
`cluster_vars`	Character vector. Default `c("Survey","Ecozon","EaId","HhId")`.

Details

`Trend` is computed **within the subset passed in**, matching the Stata code, which recomputes `sum Season / r(min)` after `keep if CropIDx == "<crop>"`. A crop whose earliest observation is not GLSS3 therefore has its trend origin at its own first wave, not the study's. This is faithful, not incidental – change it only deliberately.

Value

`data` with `.trend` (years since the subset's first wave) and `.clust` added.

Season must be a year, not a factor code

`Season` arrives from `get_household_data()` as an R **factor**, because the harmonized `.dta` labels it and `haven::as_factor()` is applied on read. Its levels span the whole GLSS series:

```
1987/88 1988/89 1990/91 1991/92 1997/98 1998/99 2004/05 2005/06
2009/10 2011/12 2012/13 2013/14 2014/15 2015/16 2016/17
```

`as.numeric()` on that returns the **level code**, not the year. For resource_extraction the four waves sit at codes 5, 6, 7, 8, 10, 11, 14, 15, so a naive conversion yields a trend of `0,1,2,3,5,6,9,10` where Stata — whose `Season` is the numeric year — gets `0,1,7,8,14,15,18,19`. The trend coefficient then comes out roughly twice too large, and every "annual percentage change" with it. The error is silent: the numbers look reasonable.

This function therefore parses the leading four-digit year out of the label (`"1997/98"` -> `1997`) whenever `Season` is not already numeric, so the unit of `.trend` is a year and the exhibits' "annual" label is true.

See Also

Other descriptive exhibits: descriptive_expand_category(), descriptive_group_summary(), descriptive_indicator_shares(), descriptive_season_year(), descriptive_specifications(), descriptive_trend_model(), descriptive_workhorse(), draw_descriptive_summary()

---

descriptive_season_year

Coerce a Season Column to a Numeric Year

Description

Returns the calendar year for a Season column, whether it arrives as a number or as a labelled factor such as "2016/17".

Usage

```
descriptive_season_year(x)
```

Arguments

x	Numeric, factor or character season column.

Details

Guards against as.numeric(factor), which silently returns the level index. See the *Season must be a year* section of descriptive_prepare().

Value

A numeric vector of four-digit years.

See Also

Other descriptive exhibits: descriptive_expand_category(), descriptive_group_summary(), descriptive_indicator_shares(), descriptive_prepare(), descriptive_specifications(), descriptive_trend_model(), descriptive_workhorse(), draw_descriptive_summary()

Examples

```
descriptive_season_year(factor(c("1997/98", "2016/17")))
```

`descriptive_specifications` *Build a Descriptive Exhibit Specification Grid*

Description

Enumerates the (treatment x crop x outcome) combinations a study's descriptive exhibits require. Mirrors `sf_model_specifications()` for the frontier layer.

Usage

```
descriptive_specifications(
  data,
  outcomes,
  treatments,
  crops = NULL,
  crop_var = "CropID",
  families = "gaussian"
)
```

Arguments

`data`	`data.frame` of the merged analysis sample, normally `study_environment$study_raw_data`.
`outcomes`	Character vector of outcome columns (Engine A), e.g. `c("Yield","Area","SeedKg", ...)`.
`treatments`	Character vector of binary treatment columns, e.g. `"OwnLnd"` or `c("extraction_any","mining_any ...)`.
`crops`	Character vector of crops to enumerate, or `NULL` (default) to take every level present in `crop_var`.
`crop_var`	Character. Column holding the crop. Default `"CropID"`.
`families`	Either a single family applied to every outcome (`"gaussian"`, the default), or a **named** character vector mapping outcome to family. See *Outcomes do not all share a model*.

Details

The land tenure study's `100_exhibits.do` hardcodes `i.OwnLnd` at every site and has no treatment loop, which is why it cannot vary its treatment without editing code. Every other study generalises to a `disagCat` column. This function makes the loop a data structure, so a study varies its treatment by passing a longer `treatments` vector.

Value

A `data.frame` with columns `treatment`, `crop`, `outcome`, `family`.

Outcomes do not all share a model

A study's Table 1 is usually estimated with more than one model, and the do-files express that as separate loops. In `land_tenure/scripts/100_exhibits.do`:

```
line  51  foreach Var of var Yield Area SeedKg HHLaborAE HirdHr FertKg PestLt
                            AgeYr YerEdu HHSizeAE Depend CrpMix
          -> reg `Var' c.Trend##i.OwnLnd            (gaussian)

line 103  foreach Var of var Female EqipMech Credit Extension EqipIrig
          -> logit `Var' c.Trend##i.OwnLnd          (binomial)
```

Both write to the same sheet, so the distinction is invisible downstream – `Equ == "Female"` and `Equ == "Yield"` sit side by side with no record that one came from a logit. Passing a named `families` vector makes it explicit and carries it into the emitted schema:

```
descriptive_specifications(
  d, outcomes = c(cont, bin), treatments = "OwnLnd",
  families = c(setNames(rep("gaussian", length(cont)), cont),
               setNames(rep("binomial",  length(bin)),  bin)))
```

See Also

Other descriptive exhibits: `descriptive_expand_category()`, `descriptive_group_summary()`, `descriptive_indicator_shares()`, `descriptive_prepare()`, `descriptive_season_year()`, `descriptive_trend_model()`, `descriptive_workhorse()`, `draw_descriptive_summary()`

---

descriptive_trend_model

Trend and Difference Tests for One Outcome

Description

Engine A's estimation step. Fits `outcome ~ trend * treatment` with clustered standard errors and returns the per-group percentage trend plus the two Wald tests the exhibits report. Replaces Stata's `reg y c.Trend##i.g, vce(cluster C)` + `testparm` + `margins ... eydx` + `nlcom (... * 100)`.

Usage

```
descriptive_trend_model(
  data,
  outcome,
  treatment = NULL,
  family = c("gaussian", "binomial"),
  weights = NULL,
  ssc = NULL,
  min_events = 10L,
  min_group_n = 30L
)
```

Arguments

data	data.frame from descriptive_prepare(), subset to one crop.
outcome	Character. Outcome column.
treatment	Character or NULL. Binary treatment column.
family	"gaussian" (default) or "binomial" for a dummy outcome.
weights	Character or NULL.
ssc	fixest::ssc() object controlling the small-sample correction. Defaults to NULL (fixest default), which needs no tuning to match the reference: point estimates and p-values both agree.
min_events	Integer. For family = "binomial", the minimum number of observations in the rarer class – overall and within each treatment group – below which no model is fitted and NULL is returned. Guards against the near-separation fits that produce nonsense trends for rare indicators inside small crops. Default 10.
min_group_n	Integer. Minimum observations in the smaller treatment group, for either family. Default 30.

Details

Emitted statistics:

trend_pct Average semi-elasticity of outcome with respect to trend, times 100 – i.e. the annual percentage change. Stata computes this as margins <g>, eydx(Trend) followed by nlcom (_b[...] * 100); here it is marginaleffects::avg_slopes() with slope = "eydx". Emitted per group and pooled.

cat_diff testparm i.g – the treatment main effect, i.e. the group level difference **at** trend == 0, not the difference in overall means. This is why an outcome can show a large raw gap yet an insignificant cat_diff.

trend_diff testparm c.Trend#i.g – the interaction.

Only p is populated for the two Wald rows, matching the sheet, where CATDif / TrendDif carry a p-value and nothing else.

Value

A data.frame with wave = "all", group, statistic, estimate, se, t, p; or NULL if the data cannot support the model.

Refusing an unestimable trend

A trend is only fitted where the data can carry it. Both guards return NULL rather than a number, and descriptive_workhorse() logs the refusal.

This is a deliberate divergence from the reference implementation, and the only one in this file. Wrapping an estimation in a swallow-all handler leaves the exported cell at whatever it held, and the consequences reach the page:

```
Pineapple Depend Trend_disagCat1   -669,487,338,753,097,984  (percent/year)
Pineapple FertKg Trend_disagCat1   -153,054,440,947,974,016
Onion     PestLt Trend_disagCat1      8,109,369,589,760
salt      x Tomatoe  Trend                              0    (fit failed)
```

Those are collapsed fits recorded as estimates. Reproducing them would be faithful and wrong: an annual percentage change of -6.7e17 is not a finding, and a crop-by-treatment trend estimated off a handful of farms should not be printed at all. Where a cell is genuinely estimable this function matches Stata exactly (14,724 parity assertions, both engines, 7 treatments, 28 crops).

Raise `min_group_n` / `min_events` to be stricter, or lower them to 0 to reproduce the Stata behaviour including its failures.

See Also

Other descriptive exhibits: `descriptive_expand_category()`, `descriptive_group_summary()`, `descriptive_indicator_shares()`, `descriptive_prepare()`, `descriptive_season_year()`, `descriptive_specifications()`, `descriptive_workhorse()`, `draw_descriptive_summary()`

`descriptive_workhorse` *Run One Descriptive Specification*

Description

Run One Descriptive Specification

Usage

```
descriptive_workhorse(
  spec_row,
  data,
  crop_var = "CropID",
  wave_var = "Surveyx",
  season_var = "Season",
  cluster_vars = c("Survey", "Ecozon", "EaId", "HhId"),
  weights = NULL,
  family = "gaussian",
  ssc = NULL,
  quiet = FALSE
)
```

Arguments

`spec_row`	One row of `descriptive_specifications()`.
`data`	`data.frame` of the merged analysis sample.
`crop_var, wave_var, season_var, cluster_vars`	Column names.
`weights`	Character or `NULL`.
`family`	Passed to `descriptive_trend_model()`. Ignored when `spec_row` carries a `family` column, which `descriptive_specifications()` emits.
`ssc`	Passed to `descriptive_trend_model()`.
`quiet`	Logical. If `FALSE` (default) a specification that yields nothing emits a message. The Stata path wrapped every specification in `cap{ }`, which swallowed failures silently – `LndAq_6` is missing from the land sheet for exactly this reason. Do not restore that behaviour.

Value

A long `data.frame`, or `NULL`.

See Also

Other descriptive exhibits: `descriptive_expand_category()`, `descriptive_group_summary()`, `descriptive_indicator_shares()`, `descriptive_prepare()`, `descriptive_season_year()`, `descriptive_specifications()`, `descriptive_trend_model()`, `draw_descriptive_summary()`

`draw_descriptive_summary`

Compute a Study's Descriptive Exhibits

Description

Runs every specification and binds the results into the long frame the table builders consume. This is the Stata-free, Excel-free replacement for a study's `100_*.do` descriptive output.

Usage

```
draw_descriptive_summary(specifications, data, study = NA_character_, ...)
```

Arguments

`specifications`	From `descriptive_specifications()`.
`data`	`data.frame`, normally `study_environment$study_raw_data`.
`study`	Character. Study name, emitted as a column.
`...`	Passed to `descriptive_workhorse()`.

Details

The returned frame is keyed, not positional. Builders filter it on `(treatment, crop, outcome, wave, group, statisti` Addressing results positionally – by row name or matrix equation – lets two outcomes collide under one label and produces a plausible wrong number rather than an error.

`attr(x, "weights")` records whether weights were applied, so a downstream table can state it rather than assume it.

Value

A long `data.frame`: `study`, `treatment`, `crop`, `outcome`, `wave`, `group`, `statistic`, `estimate`, `se`, `t`, `p`, `min`, `max`, `sd`, `n`.

See Also

Other descriptive exhibits: `descriptive_expand_category()`, `descriptive_group_summary()`, `descriptive_indicator_shares()`, `descriptive_prepare()`, `descriptive_season_year()`, `descriptive_specifications()`, `descriptive_trend_model()`, `descriptive_workhorse()`

Examples

```
## Not run:
se <- readRDS("studies/resource_extraction/data/resource_extraction_study_environment.rds")
d  <- se$study_raw_data
sp <- descriptive_specifications(
  d,
  outcomes   = c("Yield","Area","SeedKg","HHLaborAE","HirdHr","FertKg",
                 "PestLt","AgeYr","YerEdu","HHSizeAE","Depend","CrpMix"),
  treatments = c("extraction_any","mining_any","mining_comm","mining_gala",
                 "quarrying","sand","salt"))
res <- draw_descriptive_summary(sp, d, study = "resource_extraction")

## End(Not run)
```

`draw_matched_samples` *Draw matched samples (stratified bootstrap + matching)*

Description

Performs stratified resampling at the `Surveyx`, `EaId` level (excluding EaIds listed for the current bootstrap `ID`) and computes adjusted sampling weights used for matching. Then fits a matching model per `match_specifications[i, ]` using **MatchIt**, with exact matching on `Emch` and distance/link from `match_specifications`.

Usage

```
draw_matched_samples(
  i,
  data,
  match_variables_exact,
  match_variables_scaler,
  match_variables_factor,
  match_specifications,
  sample_draw_list,
  verbose = FALSE
)
```

Arguments

`i`	Integer index selecting the row of `match_specifications` to use.
`data`	A data.frame/data.table containing at least: `Surveyx`, `EaId`, `HhId`, `Mid`, `unique_identifier`, `Weight`, `Treat`, plus columns named in `Emch`, `Scle`, `Fixd`.
`match_variables_exact`	A character vector of variable names to be included in the exact match component (e.g., `"gender"`, `"region"`).
`match_variables_scaler`	A character vector of continuous or scalar variable names to include as covariates in the general match formula (e.g., `"age"`, `"income"`).

match_variables_factor
: A character vector of factor variable names to be included as dummy-coded categorical covariates in the general match formula.

match_specifications
: Data frame of matching specifications with columns boot, method, distance, and optionally link.

sample_draw_list
: Data frame where column ID identifies the bootstrap draw, and each remaining column corresponds to a Survey containing the sampled EaId.

verbose
: Logical; if TRUE, prints the chosen matching method and timing. Default FALSE.

Details

Adjusted weights are computed as $pWeight = Weight \times (alloc/allocj)$, where alloc is the pre-exclusion sum of Weight by Surveyx, EaId and allocj is the post-exclusion sum.

Exact matching is performed on variables in Emch. The distance model formula is constructed as Treat ~ Scle + Fixd. When distance == "glm", m.order = "largest" is used; otherwise "closest".

Value

A list with:

match_specifications The selected row from match_specifications.

m.out The matchit object.

md Matched data: Surveyx, EaId, HhId, Mid, unique_identifier, weights, pWeight.

df The analysis data with adjusted weights: Surveyx, EaId, HhId, Mid, unique_identifier, pWeight.

See Also

Other matching: covariate_balance(), match_sample_specifications(), treatment_effect_calculation(), treatment_effect_summary(), write_match_formulas()

---

draw_msf_estimations *Run a single MSF draw and summarize stochastic frontier results*

Description

Performs one iteration of multi-stage stochastic frontier (MSF) estimation for a given draw. The function (i) filters the analysis sample using a draw-specific exclusion list, (ii) calls msf_workhorse() to estimate production, inefficiency, and risk components by survey, and (iii) optionally builds disaggregated efficiency score summaries.

Usage

```
draw_msf_estimations(
  draw,
  drawlist,
  surveyy = FALSE,
  data,
  output_variable,
  input_variables,
  inefficiency_covariates = NULL,
  risk_covariates = NULL,
  weight_variable = NULL,
  production_slope_shifters = NULL,
  intercept_shifters = NULL,
  f,
  d,
  identifiers,
  include_trend = FALSE,
  technology_variable = NULL,
  matching_type = NULL,
  adoption_covariates = NULL,
  intercept_shifters_meta = NULL,
  disagscors_list = NULL,
  match_specifications,
  match_specification_optimal,
  match_path
)
```

Arguments

`draw`	Integer (or numeric coercible to integer). Draw iteration index. Typically `0` is used for the full baseline sample and positive integers for resampled or matched draws.
`drawlist`	A data.frame or similar object containing exclusion information by draw. Rows correspond to draws, with column `ID` identifying the draw and subsequent columns containing `EaId` values to remove from `data` for that draw.
`surveyy`	Logical; if `TRUE`, uses the survey labels stored in `data$Surveyx`. If `FALSE` (default), all observations are assigned to a synthetic survey labeled `"GLSS0"` for estimation.
`data`	A data.frame or data.table containing the estimation sample. Must include, at minimum, the columns referenced in other arguments (e.g., `output_variable`, `input_variables`, `identifiers`, `weight_variable`, technology and matching variables, and any variables used in `disagscors_list`).
`output_variable`	Character scalar. Name of the dependent (output) variable used in the production frontier.
`input_variables`	Character vector of input (production) variables entering the stochastic frontier.
`inefficiency_covariates`	Optional named list specifying variables in the inefficiency function. Typical structure: `list(Svarlist = c(...), Fvarlist = c(...))`.

`risk_covariates`
: Optional named list specifying variables in the production risk (noise) function. If `NULL`, a homoskedastic noise term is usually implied.

`weight_variable`
: Optional character scalar giving the name of the sampling weight variable in `data`. Used when computing weighted summaries (e.g., weighted means) of efficiency scores.

`production_slope_shifters`
: Character scalar naming a variable that shifts the production-function slope (e.g., technology shifter). Defaults to `NULL` to indicate no slope shifter.

`intercept_shifters`
: Optional named list of intercept shifter variables for the baseline (unmatched) sample. Typical structure: `list(Svarlist = c(...), Fvarlist = c(...))`.

`f`
: Functional form identifier passed to `msf_workhorse()` (e.g., Cobb-Douglas, translog). The exact meaning is handled by the workhorse function.

`d`
: Distributional form identifier for the inefficiency term (e.g., half-normal, truncated-normal), passed through to `msf_workhorse()`.

`identifiers`
: Character vector of variable names that uniquely identify units (e.g., household, plot, or observation IDs). These are used to merge efficiency scores back to `data` and for disaggregated summaries.

`include_trend`
: Logical; if `TRUE`, includes a technology trend variable (given by `technology_variable`) in the frontier. Defaults to `FALSE`.

`technology_variable`
: Optional character scalar naming the technology (trend) variable to be used when `include_trend = TRUE`.

`matching_type`
: Optional variable or object controlling nearest-neighbor matching; passed to `msf_workhorse()`. The expected type and structure depend on the implementation of that workhorse function.

`adoption_covariates`
: Optional named list specifying inefficiency-function variables for the matched sample (post-matching specification). Same structure as `inefficiency_covariates`.

`intercept_shifters_meta`
: Optional named list of intercept shifters for the matched sample. Same structure as `intercept_shifters`.

`disagscors_list`
: Optional character vector of variable names for which disaggregated efficiency score summaries should be computed. For each variable in this list, the function builds weighted and unweighted summaries of TE/TE0/TGR/MTE by survey, sample, and disaggregation level.

`match_specifications`
: match specifications

`match_specification_optimal`
: match specifications

`match_path`
: match path

Details

The main outputs include model estimates, mean/percentile summaries, and (optionally) disaggregated efficiency statistics by user-specified grouping variables. For draws other than zero, sample-level results are dropped to reduce storage.

1. **Draw-specific filtering:** Using `drawlist`, the function removes any `EaId` values associated with the current `draw` from `data`. This allows for bootstrap, jackknife, or matched-sample resampling.
2. **Survey-specific estimation:** After setting the `Surveyy` label (either from `Surveyx` or collapsed to `"GLSS0"`), the function loops over unique survey labels and calls `msf_workhorse()` for each. The workhorse is expected to return a list with components such as `sf_estm`, `el_mean`, `ef_mean`, `rk_mean`, `ef_dist`, `rk_dist`, `el_samp`, `ef_samp`, and `rk_samp`.
3. **Disaggregated efficiency summaries:** If `disagscors_list` is provided, the function constructs weighted and unweighted efficiency statistics (e.g., weighted mean, mean, median, mode) for TE/TE0/TGR/MTE by survey, sample, restriction status, technology, and disaggregation level. Results are stored in `res$disagscors`.
4. **Draw-specific pruning:** When `draw != 0`, sample-level objects (`el_samp`, `ef_samp`, `rk_samp`) are removed from the returned list to reduce memory usage, keeping only aggregate results.

Internally, the function uses **data.table**, **dplyr**, **tidyr**, **doBy**, and **crayon** for data manipulation and progress messages. All estimation is delegated to `msf_workhorse()`.

Value

A named list with components:

- `sf_estm` - Combined frontier parameter estimates across surveys (rows tagged with `Surveyy` and `draw`).
- `el_mean`, `ef_mean`, `rk_mean` - Mean/summary statistics for elasticities, efficiency, and risk metrics.
- `ef_dist`, `rk_dist` - Distributions of efficiency and risk quantities.
- `el_samp`, `ef_samp`, `rk_samp` - Sample-level results (only retained when `draw == 0`).
- `disagscors` - Optional disaggregated efficiency summary table if `disagscors_list` is not `NULL`; otherwise `NULL`.

If an error occurs anywhere in the pipeline, the function returns `NULL`.

`draw_msf_summary` *Summarize multi-draw Meta Stochastic Frontier (MSF) results*

Description

Aggregates results from multiple stochastic frontier analysis (SFA) / MSF draws (e.g., from a bootstrap or jackknife procedure) and computes jackknife-style summary statistics for coefficients, efficiency scores, elasticities, risk measures, and their distributions.

Usage

```
draw_msf_summary(res, technology_legend)
```

Arguments

`res` List of draw-level result objects. Each element is expected to be a list with (at least) the following components:

- `sf_estm`: coefficient-level estimates and diagnostics for naive, group, and meta-frontiers, including a `draw` index.
- `ef_mean`: aggregated efficiency statistics (e.g., TE0, TE, TGR, MTE) by sample, technology, survey, and estimation type.
- `el_mean`: aggregated elasticity statistics by input, technology, and survey.
- `rk_mean`: aggregated risk statistics, if available.
- `ef_dist`: histogram-style efficiency distributions (counts/weights) over value ranges (`binned_range_name`, `binned_range_level`).
- `rk_dist`: histogram-style risk distributions, if available.
- `disagscors`: optional disaggregated efficiency summaries by subgroup levels.
- `el_samp`, `ef_samp`, `rk_samp`: observation-level elasticities, efficiencies, and risk measures for at least the baseline draw (usually `draw == 0`).

Each component must include a `draw` column that distinguishes the baseline draw (typically `draw == 0`) from resampled draws (`draw != 0`).

`technology_legend`

Data.frame providing the mapping between numeric technology codes and their labels, typically with at least:

- `Tech`: numeric technology index used internally in MSF estimation, and
- a corresponding label column (e.g., the original `technology_variable`).

This is used to compute technology gaps (e.g., efficiency or disaggregated efficiency gaps) relative to the reference technology (smallest value in `technology_legend$Tech`).

Details

The function assumes each element of `res` is the output of a single draw from an MSF pipeline (e.g., `msf_workhorse()` followed by `draw_msf_estimations()`), containing components such as: `sf_estm`, `ef_mean`, `el_mean`, `rk_mean`, `ef_dist`, `rk_dist`, and (optionally) `disagscors`.

For each component, the function:

1. Stacks all draws using `data.table::rbindlist()`.
2. Drops non-finite values (`NA`, `Inf`, `-Inf`, `NaN`).
3. For each statistic (e.g., coefficient estimate, mean efficiency), identifies the baseline draw (`draw == 0`) and joins it with resampled draws (`draw != 0`) aggregated via `doBy::summaryBy()` to compute:
   - `Estimate.mean`: mean across non-baseline draws,
   - `Estimate.sd`: standard deviation across non-baseline draws,
   - `Estimate.length`: number of non-baseline draws.
4. Computes a jackknife-style test statistic and p-value:
   - `jack_zv = Estimate / Estimate.sd`,
   - `jack_pv = 2 * (1 - pt(|jack_zv|, df = Estimate.length))`,

   where `Estimate` is the baseline draw statistic and the distribution of resampled draws is treated as a reference distribution.

For disaggregated scores (`disagscors`), the function also:

- Computes technology-specific disaggregated efficiency gaps (level and percent) relative to the reference technology in `technology_legend`, and
- Aggregates these gaps across draws in the same jackknife fashion as for the main efficiency statistics.

Value

A list with the following components:

- `sf_estm`: Data.frame of jackknife summary statistics for frontier coefficients and diagnostics across draws, with columns such as `Estimate`, `Estimate.mean`, `Estimate.sd`, `Estimate.length`, `jack_zv`, and `jack_pv`, indexed by `Survey`, `CoefName`, `Tech`, `sample`, and `restrict`.
- `ef_mean`: Jackknife summaries for efficiency statistics (TE0, TE, TGR, MTE), by sample, technology, survey, type, estimation type, statistic, and restriction.
- `el_mean`: Jackknife summaries for elasticity statistics, by input, technology, survey, statistic, and restriction.
- `rk_mean`: (If available) jackknife summaries for risk statistics, with `type = "risk"`.
- `ef_dist`: Jackknife summaries of efficiency distributions (counts and weights) over ranges, with `stat` indicating whether the row refers to counts or weights.
- `rk_dist`: (If available) jackknife summaries of risk distributions over ranges, with `type = "risk"`.
- `disagscors`: (If available) jackknife summaries for disaggregated efficiency statistics and their gaps by subgroup level and technology.
- `el_samp`: Observation-level elasticity outputs from the first draw in `res` (typically the baseline draw).
- `ef_samp`: Observation-level efficiency scores from the first draw in `res`.
- `rk_samp`: Observation-level risk measures from the first draw in `res`.

`ers_theme` *ERS ggplot2 Theme*

Description

The shared plot theme: sans text at 9pt, no axis ticks or lines, horizontal grid only, bottom-centred legend. Applied by every `fig_*()` builder so the figures across the studies look like one publication.

Usage

```
ers_theme()
```

Value

A **ggplot2** theme object.

See Also

Other exhibit figures: `fig_covariate_balance()`, `fig_distribution()`, `fig_dsistribution()`, `fig_heterogeneity00()`, `fig_input_te()`, `fig_robustness()`, `tab_main_specification()`

exhibit_cell *Format a Mean (SD) or Estimate* SE *Cell*

Description

Formats the two cell styles used across the study manuscripts: a mean with its standard deviation in parentheses, and an estimate with significance stars and its standard error in brackets.

Usage

```
exhibit_cell(
  estimate,
  spread,
  p = NA_real_,
  digits = 2,
  style = c("mean", "estimate"),
  dagger = FALSE
)
```

Arguments

estimate	Numeric point estimate.
spread	Numeric standard deviation or standard error.
p	Numeric p-value, used only when style = "estimate". NA suppresses stars.
digits	Integer. Decimal places. Default 2.
style	"mean" gives "1.23 (4.56)"; "estimate" gives "1.23*** [4.56]".
dagger	Logical. Append a space and a dagger (U+2020) to flag a significant difference from the pooled sample.

Value

A length-one character; "" when estimate is NA.

See Also

Other exhibits: exhibit_crop_table(), exhibit_group_sizes(), exhibit_sheet_to_schema(), exhibit_stars(), exhibit_value(), exhibit_wave_table(), read_exhibit_sheet()

exhibit_crop_table *Build a Crop-by-Family Headcount Table*

Description

Builds the appendix analogues of the main tenure/exposure table: one row per crop, one column per member of a variable family, each cell a mean with its standard deviation. This is the shape of the land tenure study's Tables S1–S4 and of the equivalent crop-disaggregated appendix tables in the other studies.

Usage

```
exhibit_crop_table(
  sheet,
  variables,
  crops,
  measure = "GLSS0",
  digits = 3,
  pooled_key = "Pooled",
  pooled_label = "All crops listed"
)
```

Arguments

sheet	data.frame from read_exhibit_sheet(path, "<study>").
variables	Character vector naming the family members, in column order (e.g. c("LndOwn_1", "LndOwn_2", "LndOwn_3")).
crops	Character vector of crops, in row order. The pooled key is appended automatically and must not be included.
measure	Character. Value of mesure to read. Default "GLSS0".
digits	Integer. Decimal places. Default 3.
pooled_key	Character. Value of crop holding the pooled row.
pooled_label	Character. Row label for the pooled row.

Details

These tables are the *same sheet* as the pooled main table, sliced by crop instead of restricted to "Pooled". In the land tenure study:

Table	variables	Content
S1	LndOwn_1..3	ownership status
S2	LndAq_1..5	mode of acquisition
S3	LndRgt_1..4	ownership rights
S4	ShrCrpCat_1..3	sharecropping intensity

The pooled row is emitted last, labelled pooled_label, and is the same crop == pooled_key row the main table reports.

measure defaults to "GLSS0", the headcount pooled across the waves in which the module was administered, which is what the appendix tables report.

Value

A data.frame with label, header and c1..cN, ready for a flextable builder.

See Also

read_exhibit_sheet(), exhibit_wave_table()

Other exhibits: exhibit_cell(), exhibit_group_sizes(), exhibit_sheet_to_schema(), exhibit_stars(), exhibit_value(), exhibit_wave_table(), read_exhibit_sheet()

exhibit_group_sizes *Group Sizes from a Tidy* means *Sheet*

Description

Returns the analysis-sample sizes for the pooled sample and each treatment group, read from the workbook rather than hardcoded in a study script.

Usage

```
exhibit_group_sizes(means, groups, equ = "Female", crop = "Pooled")
```

Arguments

means	data.frame from read_exhibit_sheet(path, "means").
groups	Character vector of group suffixes as they appear in Coef, e.g. c("Pooled", "OwnLnd0", "OwnLnd1").
equ	Character. Any outcome present for every group; used only to locate the Mean_<group> rows. Defaults to "Female".
crop	Character. Crop to read. Defaults to "Pooled".

Details

These count **rows of the analysis sample** (CropIDx == "Pooled"), not distinct farmers. For the land tenure study the farmer key (Surveyx, EaId, HhId, Mid) is *not* unique within the pooled sample – 28,411 distinct keys against 35,185 rows, because a farmer may appear in more than one season. Prose should therefore say "observations of farm households", not "farm households".

Value

A named numeric of group sizes, named by groups.

See Also

Other exhibits: exhibit_cell(), exhibit_crop_table(), exhibit_sheet_to_schema(), exhibit_stars(), exhibit_value(), exhibit_wave_table(), read_exhibit_sheet()

exhibit_sheet_to_schema *Translate a Stata Exhibit Sheet into the Engine Schema*

Description

Converts a sheet from read_exhibit_sheet() into the same long, keyed frame draw_descriptive_summary() emits, so that the Stata workbook and the R engine speak one vocabulary and every downstream table builder can consume either.

Usage

```
exhibit_sheet_to_schema(
  sheet,
  group_tag = "disagCat",
  treatment = NA_character_,
  trend_statistic = c("trend_pct", "change_pp")
)
```

Arguments

sheet	A data.frame from read_exhibit_sheet().
group_tag	Character. The treatment tag used in the sheet's Coef strings. Every do-file writes gen disagCat = \disag'and names its matrix rows from that, so this is usually the
treatment	Character. Value for the emitted treatment column.
trend_statistic	"trend_pct" or "change_pp". See *Trend flavor*. Applies to the Engine B (study) layout only.

Details

The workbook addresses results by matrix position; the engine keys them. The mapping:

Mean_<tag>0 / Mean_<tag>1 / Mean_Pooled	statistic = "mean", wave = "all"
GLSS6_<tag>0 (etc.)	statistic = "mean", wave = "GLSS6"
Trend_<tag>0 (etc.)	statistic = "trend_pct", wave = "all"
CATDif	statistic = "cat_diff", group = NA
TrendDif	statistic = "trend_diff", group = NA

Rows that carry no result – <wave>_miss, r1, _ – are dropped.

Value

A data.frame with treatment, crop, outcome, wave, group, statistic, estimate, se, t, p, min, max, sd, n.

Why sd and n are dropped on the model rows

The do-files build the model rows from mat A = r(table)', whose first eight columns are b, se, t, p, ll, ul, df, crit. They are then given the column names Beta SE Tv Pv Min Max SD N. So on a Trend_* / CATDif / TrendDif row the SD column actually holds the **degrees of freedom** and N holds the **critical value** – which is why those rows show N = 1.959964 throughout the sheets. Mapping them to sd and n would silently pass off a z-value as a sample size, so this function sets both to NA there and keeps them only on the tabstat-derived mean rows, where they mean what they say.

Trend flavor

Engine B's trend is not the same estimator in every study, and the sheet does not record which was used (see descriptive_indicator_shares()). Pass trend_statistic = "trend_pct" for a margins ... eydx semi-elasticity (resource_extraction) or "change_pp" for a wave difference in percentage points (land_tenure). Getting it wrong mislabels percent as percentage points.

See Also

read_exhibit_sheet(), draw_descriptive_summary()

Other exhibits: exhibit_cell(), exhibit_crop_table(), exhibit_group_sizes(), exhibit_stars(), exhibit_value(), exhibit_wave_table(), read_exhibit_sheet()

---

exhibit_stars *Significance Stars for a P-Value*

Description

Significance Stars for a P-Value

Usage

```
exhibit_stars(p, levels = c(`***` = 0.01, `**` = 0.05, `*` = 0.1))
```

Arguments

p	Numeric p-value. NA returns "".
levels	Named numeric of thresholds, highest first. Defaults to c("***" = 0.01, "**" = 0.05, "*" = 0.10).

Value

A length-one character; "" when not significant or NA.

See Also

Other exhibits: exhibit_cell(), exhibit_crop_table(), exhibit_group_sizes(), exhibit_sheet_to_schema(), exhibit_value(), exhibit_wave_table(), read_exhibit_sheet()

---

exhibit_value *Extract a Single Value from a Tidy Exhibit Sheet*

Description

Looks up exactly one value from a sheet returned by read_exhibit_sheet(), given a set of key/value pairs identifying the row.

Usage

```
exhibit_value(data, keys, col)
```

Arguments

data	data.frame from read_exhibit_sheet().
keys	Named list of column/value pairs identifying one row, e.g. list(Equ = "Yield", CropIDx = "Pooled", Coef = "Mean_Pooled").
col	Character. Name of the column to return ("Beta", "SE", "Pv", "SD", "N", ...).

Details

Returns NA_real_ when no row matches, which lets a table builder leave a cell blank for a quantity a study does not estimate. **Errors** when more than one row matches: a duplicate key means the sheet is malformed, and silently taking the first match is how a lookup returns a plausible but wrong number. See the *Duplicate rows* section of read_exhibit_sheet().

Value

A length-one numeric, or NA_real_ if no row matches.

See Also

read_exhibit_sheet()

Other exhibits: exhibit_cell(), exhibit_crop_table(), exhibit_group_sizes(), exhibit_sheet_to_schema(), exhibit_stars(), exhibit_wave_table(), read_exhibit_sheet()

Examples

```
## Not run:
m <- read_exhibit_sheet("studies/land_tenure/output/land_tenure_results.xlsx")
exhibit_value(m, list(Equ = "Yield", CropIDx = "Pooled",
                      Coef = "Mean_Pooled"), "Beta")

## End(Not run)
```

---

exhibit_wave_table *Build a Variable-by-Wave Table*

Description

Builds the pooled main tenure/exposure table: one row per variable, one column per survey wave, plus a trend column. This is the shape of the land tenure study's Table 2.

Usage

```
exhibit_wave_table(
  sheet,
  map,
  waves,
  trend = "Trend",
  crop = "Pooled",
  digits = 3
)
```

Arguments

sheet	data.frame from read_exhibit_sheet(path, "<study>").
map	data.frame with columns label, header (1 marks a bold section row with no values) and Variable (NA on header rows).

waves	Character vector of mesure values to emit as mean columns, in order (e.g. c("GLSS6", "GLSS7")).
trend	Character or NULL. mesure value for a trailing estimate/standard-error column. Default "Trend".
crop	Character. Crop to read. Default "Pooled".
digits	Integer. Decimal places. Default 3.

Value

A data.frame with label, header and c1..cN.

See Also

exhibit_crop_table()

Other exhibits: exhibit_cell(), exhibit_crop_table(), exhibit_group_sizes(), exhibit_sheet_to_schema(), exhibit_stars(), exhibit_value(), read_exhibit_sheet()

---

fig_covariate_balance *Covariate Balance Figure*

Description

Plots standardised mean differences and variance ratios across the candidate matching algorithms – the evidence behind the choice of matched sample. Balance is best where the standardised difference is near zero and the variance ratio near one.

Usage

```
fig_covariate_balance(
  colset = c("darkgreen", "orange", "blue"),
  study_environment
)
```

Arguments

colset	Character vector of colours.
study_environment	The study environment list.

Value

A **ggplot2** object.

See Also

Other exhibit figures: ers_theme(), fig_distribution(), fig_dsistribution(), fig_heterogeneity00(), fig_input_te(), fig_robustness(), tab_main_specification()

`fig_distribution` *Score Distribution Figure*

Description

Plots the distribution of the efficiency scores by treatment group.

Usage

```
fig_distribution(dataFrq, colset = c("orange", "darkgreen"), study_environment)
```

Arguments

`dataFrq`	`data.frame` of scores to plot.
`colset`	Character vector of colours.
`study_environment`	The study environment list.

Value

A **ggplot2** object.

Renamed from `fig_dsistribution()`

The original was misspelled. `fig_dsistribution()` is retained as a deprecated alias so the six studies that still call it keep working; it warns and forwards here. Use `fig_distribution()` in new code, and drop the alias once no study calls it.

See Also

Other exhibit figures: `ers_theme()`, `fig_covariate_balance()`, `fig_dsistribution()`, `fig_heterogeneity00()`, `fig_input_te()`, `fig_robustness()`, `tab_main_specification()`

`fig_dsistribution` *Score Distribution Figure (deprecated spelling)*

Description

Deprecated misspelling of `fig_distribution()`. Warns and forwards.

Usage

```
fig_dsistribution(
  dataFrq,
  colset = c("orange", "darkgreen"),
  study_environment
)
```

Arguments

dataFrq	data.frame of scores to plot.
colset	Character vector of colours.
study_environment	The study environment list.

Details

Retained because six studies still call it: ag_services, disability, education, financial_inclusion, resource_extraction, time_poverty. Delete once none do.

Value

A **ggplot2** object.

See Also

Other exhibit figures: ers_theme(), fig_covariate_balance(), fig_distribution(), fig_heterogeneity00(), fig_input_te(), fig_robustness(), tab_main_specification()

---

fig_heterogeneity00 *Heterogeneity Figure*

Description

Plots the treatment gap across farmer characteristics, crops and regions – the study's heterogeneity panel.

Usage

```
fig_heterogeneity00(
  res,
  y_title,
  colset = c("orange", "darkgreen", "blue"),
  study_environment
)
```

Arguments

res	data.frame from tab_main_specification().
y_title	Character. Y-axis title.
colset	Character vector of colours.
study_environment	The study environment list.

Value

A **ggplot2** object.

See Also

Other exhibit figures: ers_theme(), fig_covariate_balance(), fig_distribution(), fig_dsistribution(), fig_input_te(), fig_robustness(), tab_main_specification()

fig_input_te *Input and Output Treatment-Effect Figure*

Description

Plots the matched-sample gaps in input use and output between treatment groups. Also writes `output/figure_data/input_TE_data.csv`, which `exhibit_helpers_tables.R` reads for the `fig1_range()` inline lookups – so this builder must run before the article is knitted.

Usage

```
fig_input_te(
  y_title,
  tech_lable,
  colset = c("orange", "darkgreen", "blue"),
  study_environment
)
```

Arguments

`y_title`	Character. Y-axis title.
`tech_lable`	Character. Label for the technology/treatment groups.
`colset`	Character vector of colours.
`study_environment`	The study environment list.

Value

A **ggplot2** object.

See Also

Other exhibit figures: `ers_theme()`, `fig_covariate_balance()`, `fig_distribution()`, `fig_dsistribution()`, `fig_heterogeneity00()`, `fig_robustness()`, `tab_main_specification()`

fig_robustness *Robustness Figure*

Description

Plots the treatment gap across alternative specifications – functional form, inefficiency distribution, score aggregation, matching algorithm.

Usage

```
fig_robustness(
  y_title,
  res_list,
  colset = c("orange", "darkgreen"),
  study_environment
)
```

Arguments

y_title	Character. Y-axis title.
res_list	Character vector of paths to estimation .rds files.
colset	Character vector of colours.
study_environment	The study environment list.

Value

A **ggplot2** object.

See Also

Other exhibit figures: ers_theme(), fig_covariate_balance(), fig_distribution(), fig_dsistribution(), fig_heterogeneity00(), fig_input_te(), tab_main_specification()

---

get_crop_area_list *Extract Matching Crop Area Columns from a Dataset*

Description

Identifies and returns the column names in a dataset corresponding to crop area variables for a specified set of crops. The function looks for column names that start with "Area_" and match any of the crops provided in selected_crops.

Usage

```
get_crop_area_list(
  data,
  selected_crops = c("Beans", "Cassava", "Cocoa", "Cocoyam", "Maize", "Millet", "Okra",
    "Palm", "Peanut", "Pepper", "Plantain", "Rice", "Sorghum", "Tomatoe", "Yam")
)
```

Arguments

data	A data.frame or data.table containing crop-related variables. Column names are expected to include fields prefixed with "Area_", such as "Area_Maize".
selected_crops	A character vector specifying the crop names to filter. Defaults to a common set of crops including "Beans", "Cassava", "Cocoa", "Cocoyam", "Maize", "Millet", "Okra", "Palm", "Peanut", "Pepper", "Plantain", "Rice", "Sorghum", "Tomatoe", and "Yam".

Value

A character vector containing the names of columns in data that correspond to the specified crop area variables (e.g., "Area_Maize", "Area_Rice"). Returns an empty vector if no matching columns are found.

See Also

Other helpers: compute_jenks_binned_shares(), harmonized_data_prep()

`get_household_data` *Download and Load Harmonized Household Data from the GHAgricProductivityLab Repository*

Description

Retrieves a harmonized household- or farm-level dataset from the **GHAgricProductivityLab** GitHub repository using **piggyback**, stores it in a package-specific cache directory, and returns the dataset as a `data.frame`. The file is downloaded only once and reused from the local cache on future calls.

Usage

```
get_household_data(
  dataset = "harmonized_crop_farmer_data",
  github_token = NULL,
  force = FALSE
)
```

Arguments

`dataset`	Character string. Base name of the dataset to retrieve (without file extension). Must correspond to a `.dta` file in the `hh_data` GitHub release (e.g., `"harmonized_crop_farmer_dat`
`github_token`	Optional GitHub personal access token (PAT). If `NULL`, the function checks the environment variable `GHProdLab_TOKEN`. If that is also missing, the piggyback download will use default authentication behavior.
`force`	force re download

Details

The function downloads a Stata `.dta` file associated with the chosen dataset from the GitHub release labeled `hh_data`. It uses the package-specific cache directory determined by:

```
tools::R_user_dir("GHAgricProductivityLab", which = "cache")
```

If the file already exists locally, it is not re-downloaded.

GitHub Authentication

- If `github_token` is supplied, it is used.
- Otherwise, the function looks for environment variable `GHProdLab_TOKEN`.
- If neither is available, the function falls back to default GitHub credentials (e.g., from `gh` CLI or cached credentials).

Value

A `data.frame` containing the requested harmonized dataset.

harmonized_data_prep *Prepare Data for Agricultural Productivity Analysis*

Description

Cleans and transforms a dataset by creating new variables, applying log transformations, converting selected variables to factors or characters, and recoding education levels. The function is designed to standardize inputs for further analysis of agricultural productivity.

Usage

```
harmonized_data_prep(data)
```

Arguments

data	A data.frame or data.table containing household- and farm-level variables such as weights, demographic information, and agricultural inputs.

Value

A cleaned and transformed data.frame or data.table with additional variables ready for analysis.

See Also

Other helpers: compute_jenks_binned_shares(), get_crop_area_list()

match_sample_specifications *Draw / Match Sample Specifications*

Description

Generates (i) a draw list by sampling EaId within each unique Survey group and (ii) a grid of matching specifications for each bootstrap draw.

Usage

```
match_sample_specifications(number_of_draws = NULL, data, myseed = NULL)
```

Arguments

number_of_draws	Integer. The number of draws to perform per Survey.
data	A data.frame or data.table containing at least the columns Survey and EaId.
myseed	Integer. Seed for random number generation (default 03242025).

Details

Draw list: For each unique value of `Survey`, the function samples `number_of_draws` `EaId` values with replacement and prepends a 0 row (ID = 0) for the baseline. The result is then spread to wide format with one column per `Survey`.

Matching specs: For each draw `ID`, creates a set of matching model specifications that include:

- Nearest neighbor with distances: `"euclidean"`, `"scaled_euclidean"`, `"mahalanobis"`, `"robust_mahalanobis"`.
- Nearest neighbor with distance `"glm"` and links: `"logit"`, `"probit"`, `"cloglog"`, `"cauchit"`.

An `ARRAY` index is added for convenience.

Value

A list with two elements:

`m.specs` A data.frame of matching specifications with columns `boot`, `method`, `distance`, `link`, `ARRAY`.

`drawlist` A data.frame in wide format where each `Survey` is a column and rows correspond to draw `ID` (0:`number_of_draws`).

Note

Requires that `data` contain `Survey` and `EaId`. `tidyr::spread()` is used for wide reshaping (consider `tidyr::pivot_wider()` in new code).

See Also

Other matching: `covariate_balance()`, `draw_matched_samples()`, `treatment_effect_calculation()`, `treatment_effect_summary()`, `write_match_formulas()`

`msf_workhorse` *Meta stochastic frontier (MSF) workhorse*

Description

Performs Meta Stochastic Frontier (MSF) analysis in several stages:

1. **Naive SF:** Fits a pooled stochastic frontier and computes baseline technical efficiency (TE) measures.
2. **Group SF:** If a technology variable `technology_variable` is supplied, splits the sample into technology groups and fits group-specific SF models.
3. **Meta SF (unmatched and matched):** Constructs a meta-frontier using fitted values from group models and, optionally, nearest-neighbor matching specifications. From this, it derives technology gap ratios (TGR) and meta-technical efficiency (MTE).
4. **Summaries and distributions:** Produces weighted and unweighted summaries and distributional statistics for efficiency, elasticity, and risk across technologies, samples (unmatched/matched), and surveys.

Usage

```
msf_workhorse(
  data,
  output_variable,
  input_variables,
  inefficiency_covariates = NULL,
  risk_covariates = NULL,
  weight_variable = NULL,
  production_slope_shifters = NULL,
  intercept_shifters = NULL,
  f,
  d,
  identifiers,
  include_trend = FALSE,
  technology_variable = NULL,
  matching_type = NULL,
  adoption_covariates = NULL,
  intercept_shifters_meta = NULL,
  match_path,
  match_specifications,
  match_specification_optimal
)
```

Arguments

`data`	A data.frame or data.table containing the estimation sample. Must include the dependent variable `output_variable`, the inputs in `input_variables`, the weight variable `weight_variable`, unique ID variables in `identifiers`, any inefficiency and risk covariates in `inefficiency_covariates` and `risk_covariates`, the technology variable `technology_variable` (if used), and any variables required by matching objects on disk (e.g., `"Surveyx"`, `"EaId"`, `"HhId"`, `"Mid"`, `"unique_identifier"`).
`output_variable`	Character scalar. Name of the dependent (output) variable used in the production frontier.
`input_variables`	Character vector of input variable names (e.g., land, labor, capital) passed through to `sf_workhorse`.
`inefficiency_covariates`	Optional named list specifying variables in the inefficiency function for the naive and group frontiers. Typical structure: `list(scalar_variables = c(...), factor_variables = c(...))`.
`risk_covariates`	Optional named list specifying variables in the production risk (noise) function for the naive and group frontiers. Same structure as `inefficiency_covariates`. If `NULL`, a homoskedastic noise term is usually implied.
`weight_variable`	Character scalar. Name of the sampling/observation weight variable in `data`. Observations with zero weight are dropped.
`production_slope_shifters`	Character scalar giving the name of a slope-shifter variable in `data` (e.g., technology shifter, policy dummy). Defaults to `NULL` for no slope shifter.

intercept_shifters
: Optional named list of intercept shifter variables for the naive and group frontiers. Typical structure: `list(scalar_variables = c(...), factor_variables = c(...))`, passed to `sf_workhorse`.

f
: Functional form index for the production frontier (e.g., Cobb-Douglas, translog, quadratic). The index is passed to `sf_workhorse` and ultimately to the internal form-selection utilities (e.g., `sf_functional_forms`).

d
: Distributional form index for the inefficiency term (e.g., half-normal, truncated-normal). Passed to `sf_workhorse`.

identifiers
: Character vector of variable names that uniquely identify observations (e.g., `c("unique_identifier","Survey","CropID","HhId","EaId","Mid")`). These IDs are used when merging scores and summarizing by technology or sample.

include_trend
: Logical; if `TRUE`, the last element in `input_variables` is treated as a trend/technology variable in the production function. Passed through to `sf_workhorse`. Defaults to `FALSE`.

technology_variable
: Optional character scalar naming the technology variable (e.g., region, system, period) used to define groups for group SF and meta-frontier estimation. If `NULL`, no technology groups are formed and only the naive frontier and TE are computed.

matching_type
: Optional character scalar controlling which nearest-neighbor matching specifications to use for meta-frontier estimation. When not `NULL`, the function reads matching specifications and matched samples from `"results/matching/"` and related RDS files. Setting `matching_type = "optimal"` restricts attention to the subset of matching specifications labeled as optimal.

adoption_covariates
: Optional named list specifying inefficiency-function covariates for the meta-frontier (matched/unmatched) estimation. If `NULL`, defaults to `inefficiency_covariates`.

intercept_shifters_meta
: Optional named list of intercept shifters for the meta-frontier estimation. If `NULL`, defaults to `intercept_shifters`.

match_path
: match path

match_specifications
: match specifications

match_specification_optimal
: match specifications

Details

The function is designed as a high-level orchestrator around `sf_workhorse`, assembling naive, group-level, and meta-frontier results into a unified output structure suitable for MSF analysis.

The workflow can be summarized as:

- **Naive SFA (TE0):** Calls `sf_workhorse` on the full sample to obtain baseline efficiencies (`teBC`, `teJLMS`, `teMO`). If no technology variable is specified (`technology_variable = NULL`), these naive efficiencies are the final scores and only TE-related summaries are produced.
- **Group SFA (TE):** When `technology_variable` is provided, the sample is recoded into numeric technology groups (`Tech`), and `sf_workhorse` is applied separately to each group. Group-level efficiencies are combined into TE measures by group and survey.

- **Meta-frontier (TGR and MTE):** Using fitted group-frontier values (`Yhat`) from the restricted models, the function fits meta-frontiers (unmatched and, optionally, matched) again via `sf_workhorse`. Technology gap ratios (TGR) are derived from these meta-frontiers, and meta-technical efficiency (MTE) is computed as `TE * TGR`.
- **Matching layer (optional):** If `matching_type` is given, the function loads matching specifications (`mspecs`, `mspecs_optimal`) and corresponding matched samples from disk (e.g., `"results/matching/"`), fits additional meta-frontiers by matching specification, and augments the set of scores and summaries with these matched samples.
- **Summaries and distributions:**

  Scores: TE0, TE, TGR, and MTE are reshaped into long form and aggregated to compute weighted/unweighted means, medians, modes, and distribution histograms (both count-based and weight-based) across surveys, samples, technologies, and estimation types. When `technology_variable` is not `NULL`, the function also computes technology gaps (level and percent) relative to the minimum technology in `technology_legend`.

  Elasticities: Elasticities from the underlying SF models (naive, group, meta) are combined and summarized in a similar way, including technology-gap metrics for elasticities when `technology_variable` is provided.

  Risk: Risk measures derived from `sf_workhorse` are combined, summarized, and used to build distributional statistics analogous to those for efficiency.
- **LR tests:** When `technology_variable` is not `NULL`, the function builds likelihood ratio test statistics comparing a naive pooled frontier to the combination of group and meta-frontiers, storing these as rows with `CoefName = "LRT"` in the `sf_estm` output.

Throughout, the function uses **dplyr**, **tidyr**, **data.table**, **doBy**, and related helper functions for reshaping and summarizing the output of `sf_workhorse`.

Value

A named list with the following components:

- `sf_estm`: Data.frame of coefficient-level summaries from all naive, group, and meta-frontiers (including LR tests), augmented with technology identifiers (`Tech`) and `sample` labels (e.g., `"unmatched"`, matching-spec names).
- `ef_mean`: Data.frame of aggregated efficiency statistics (TE0, TE, TGR, MTE), including weighted/unweighted means, medians, and modes, by survey, sample, technology, type, estimation type, and restriction status. When `technology_variable` is supplied, also includes efficiency gap levels and percentages.
- `ef_dist`: Data.frame of efficiency distributions (histogram counts and weights) over efficiency ranges, by survey, sample, technology, type, estimation type, and restriction.
- `rk_dist`: Data.frame of risk distributions (histogram counts and weights) analogous to `ef_dist`, with `type = "risk"`.
- `el_mean`: Data.frame of aggregated elasticity statistics (weighted/unweighted means, medians, modes) by input, survey, sample, technology, and restriction, including elasticity-gap measures when `technology_variable` is provided.
- `rk_mean`: Data.frame of aggregated risk statistics (weighted and unweighted) by survey, sample, technology, and restriction, with risk-gap measures when `technology_variable` is provided.
- `el_samp`: Data.frame of observation-level elasticities for all models and samples (excluding the synthetic `"GLSS0"` survey rows).

- `ef_samp`: Data.frame of observation-level efficiency scores (TE0, TE, TGR, MTE) including IDs, weights, technologies, and sample labels.
- `rk_samp`: Data.frame of observation-level risk measures for all models and samples (excluding the synthetic `"GLSS0"` survey rows).

See Also

Other frontier analysis: `sf_workhorse()`

`read_exhibit_sheet` *Read a Tidy Exhibit Sheet from a Study Results Workbook*

Description

Reads one of the machine-readable ("tidy") sheets written by a study's `100_exhibits.do` / `100_FIGTAB_*.do` into a `data.frame`, drops the display-formula spill columns, and applies the canonical column names.

Usage

```
read_exhibit_sheet(path, sheet = "means", layout = c("auto", "means", "study"))
```

Arguments

`path`	Character. Path to the `.xlsx` results workbook.
`sheet`	Character. Sheet name. See *Sheet naming across studies*.
`layout`	Character. Which column naming to apply. `"auto"` (default) detects from the sheet's own headers; `"means"` and `"study"` force it.

Details

Each study's results workbook carries two kinds of sheet:

- **Tidy value sheets**, written by Stata's `export excel ... firstrow(variables)`. These contain values, one row per estimated quantity, and are the source of truth for the manuscript tables.
- **Display sheets** (`Table1`, `Table2`, ...), hand-built with formulas that reference the tidy sheets. These cache `#N/A` once Stata rewrites the tidy data with `sheetmodify` and must not be read.

Only the first eleven columns of a tidy sheet are real; anything beyond is spill from adjacent display formulas (read by **readxl** as `...12`, `***`, `0.00 (0.00)` and similar) and is discarded.

The two recognised layouts are:

```
means   CropIDx, Equ, Coef, Beta, SE, Tv, Pv, Min, Max, SD, N
<study> Variable, crop, mesure, Beta, SE, Tv, Pv, Min, Max, SD, N
```

In the `means` layout, `Equ` is the outcome variable, `Coef` the estimated quantity (`Mean_<group>`, `Trend_<group>`, `CATDif`, `TrendDif`, `<wave>_<group>`), and `CropIDx` the crop (`"Pooled"` for all crops).

Value

A `data.frame` with eleven canonically named columns.

Duplicate rows

Some workbooks contain duplicate `(Equ, CropIDx, Coef)` keys, from a do-file that hardcoded `mat roweq A = Female` inside the loop over treatment dummies: those rows land on the `means` sheet tagged `Equ == "Female"` and collide with the real `Female` outcome rows. Check for it in any workbook you read here. `exhibit_value()` errors on duplicates rather than silently returning the first match, which is what makes the collision visible at all.

Sheet naming across studies

Sheet names are **not** consistent, because each study's do-file names them from its own treatment loop:

Study	Engine A sheet(s)
land_tenure	`means` (one treatment)
resource_extraction	`Means_extraction_any`, `Means_mining_any`, `Means_mining_comm`, `Means_mining_gala`, `Means_`

`layout = "auto"` therefore does **not** key off the sheet name. It inspects the first three column headers as written by Stata (`CropIDx, Equ, Coef` for Engine A; `Variable, crop, mesure` for Engine B) and picks accordingly. Keying off the name would silently apply the wrong column names to `Means_extraction_any`.

See Also

`exhibit_value()`, `exhibit_stars()`, `exhibit_group_sizes()`

Other exhibits: `exhibit_cell()`, `exhibit_crop_table()`, `exhibit_group_sizes()`, `exhibit_sheet_to_schema()`, `exhibit_stars()`, `exhibit_value()`, `exhibit_wave_table()`

`run_only_for` *Restrict Execution of an R Script to Allowed SLURM Job Conditions*

Description

`run_only_for()` controls when an R script is allowed to run based on:

- the SLURM array index (`SLURM_ARRAY_TASK_ID`)
- the SLURM job name (`SLURM_JOB_NAME`)
- whether the script is being run locally on Windows

The function is designed for workflows where multiple R scripts run in a single SLURM array job; each script decides independently whether it should run.

Usage

```
run_only_for(id = NULL, run_on_windows = TRUE, allowed_jobnames = NULL)
```

Arguments

id	Integer or NULL. Expected SLURM array index, or NULL to skip checking (useful for Windows/local runs).
run_on_windows	Logical. When TRUE (default), do not restrict when running on Windows.
allowed_jobnames	Character vector of SLURM job names allowed to run this script. If NULL, job-name filtering is skipped.

Details

Execution logic:

1. **Windows override** If running on Windows and run_on_windows = TRUE, return immediately.
2. **Allowed job names** If allowed_jobnames is set, the script runs only when SLURM job name is in that vector.
3. **Array ID restriction**
   - If id = NULL, skip array-index filtering.
   - If numeric, match against SLURM_ARRAY_TASK_ID.

Any violation -> quit(save = "no").

Value

Invisibly returns NULL when allowed; otherwise exits R.

---

sf_workhorse *Stochastic frontier workhorse*

Description

Fits a stochastic frontier model for a given production specification and, when monotonicity is sufficiently violated, re-estimates a shape-constrained (restricted) frontier using minimum-distance methods. The function wraps the underlying sfaR estimation routines and a set helper utilities (e.g., sf_functional_forms(), equation_editor(), fit_organizer(), micEcon::translogEla(), curvature/monotonicity checks) to produce:

1. Unrestricted stochastic frontier estimates and diagnostics.
2. Shape-constrained estimates that enforce monotonicity (and related regularity conditions) when needed.
3. Observation-level efficiencies, fitted values, and risk measures.
4. Observation-level elasticities, including a "returns-to-scale" term.

Usage

```
sf_workhorse(
  data,
  output_variable,
  input_variables,
  weight_variable = NULL,
  d,
  f,
  production_slope_shifters = NULL,
  intercept_shifters = NULL,
  inefficiency_covariates = NULL,
  risk_covariates = NULL,
  identifiers,
  include_trend = FALSE
)
```

Arguments

data	A data.frame or data.table containing all variables required for estimation, including the dependent variable output_variable, inputs input_variables, the weight variable weight_variable (if used), the unique ID variables listed in identifiers, and any variables referenced in production_slope_shifters, intercept_shifters, inefficiency_covariates, or risk_covariates.
output_variable	Character scalar. Name of the dependent/output variable in data used for the production frontier.
input_variables	Character vector of input variable names (e.g., land, labor, capital). The order of variables in input_variables determines how they map into the generic input labels I1, I2, ... used in the functional-form utilities.
weight_variable	Optional character scalar. Name of the sampling/observation weight variable. If NULL, all observations are given unit weight.
d	Distributional form index for the inefficiency term. This is passed to sf_functional_forms() and ultimately to sfacross() (e.g., to select half-normal, truncated-normal, etc.). The exact mapping is determined by sf_functional_forms().
f	Functional form index for the production frontier (e.g., Cobb-Douglas, translog, quadratic), used to select from the list returned by sf_functional_forms(). The name of the chosen form (e.g., "CD", "TL", "QD", "LN") influences logging of variables and regularity checks.
production_slope_shifters	Character scalar giving the name of a slope-shifter variable in data (e.g., technology shift, policy dummy). Defaults to NULL for no slope shifter and is passed to equation_editor().
intercept_shifters	Optional named list of intercept shifter variables for the production function. Typical structure: list(scalar_variables = c(...), factor_variables = c(...)), where the specific interpretation is handled inside equation_editor().
inefficiency_covariates	Optional named list describing the inefficiency-function covariates. Typical

structure: `list(scalar_variables = c(...), factor_variables = c(...))`, passed as `uhet` and `muhet` to `sfacross()` when non-`NULL`.

`risk_covariates`
: Optional named list describing the production-risk (noise) covariates. When non-`NULL`, used as `vhet` in `sfacross()` to allow heteroskedasticity of the noise term.

`identifiers`
: Character vector of variable names that uniquely identify observations (e.g., `c("unique_identifier","Survey","CropID","HhId","EaId","Mid")`). These IDs are carried through to efficiency, elasticity, and risk outputs.

`include_trend`
: Logical; if `TRUE`, the last element in `input_variables` is treated as a trend/technology variable and handled accordingly in the functional-form and elasticity calculations. Defaults to `FALSE`.

Details

The function proceeds in several steps:

1. **Setup and functional form selection:** Using `sf_functional_forms()`, the function determines the appropriate production functional form and distributional specification based on the number of inputs (`number_of_inputs`), the `f` and `d` indices, and the `include_trend` flag. This includes whether the model is specified in logs (e.g., CD/TL/GP/TP) or levels.
2. **Variable construction and equation building:** Inputs in `input_variables` are mapped to generic labels (`I1`, `I2`, ...) and their log transforms are created when needed. The outcome variable is set to `Y` or `lnY`. The function then calls `equation_editor()` to construct the production, inefficiency, and risk equations used by `sfacross()`.
3. **Unrestricted stochastic frontier estimation:** The function iterates over a grid of optimization methods (`nr`, `nm`, `bfgs`, `bhhh`, ...) and tolerance settings, calling `sfacross()` until a successful convergence is reported. It extracts observation-level efficiencies via `sfaR::efficiencies()`, constructs fitted values, and builds an observation-level risk metric based on squared deviations from mean output.
4. **Elasticities and regularity checks:** Using `fit_organizer()` and `micEcon::translogEla()`, the function computes elasticities and returns-to-scale-type measures, and evaluates monotonicity and curvature via `micEcon::translogCheckMono()` and `micEcon::translogCheckCurvature()` (or a coefficient-sign check for CD/LN forms).
5. **Shape-constrained frontier (if needed):** If the monotonicity measure `mono` falls below 0.80, the function constructs a set of linear restrictions using `micEcon::translogMonoRestr()` and solves a quadratic programming problem (via `quadprog::solve.QP` and fall-back matrix inversions using **Matrix**, **MASS**, **corpcor**) to obtain constrained coefficients. For TL/QD forms a constrained frontier (`lcFitted`) is computed with `micEcon::translogCalc()`; for CD/LN forms, constrained coefficients are obtained via a SEM representation using **lavaan**.
6. **Re-estimation under constraints:** Given the constrained frontier, the function rebuilds the production equation with `equation_editor()` and re-estimates a stochastic frontier (`sfc`) using the same optimization grid. Constrained efficiencies, risks, and elasticities are computed and summarized, and regularity checks are repeated.

Unrestricted and restricted summaries are then stacked, with a `restrict` flag (`"Unrestricted"` vs. `"Restricted"`), for downstream comparison.

Value

A named list with four data.frames:

- `sf`: Coefficient-level results combining unrestricted and, when applicable, restricted SFA estimates, plus monotonicity/curvature diagnostics. Columns include (at minimum) `CoefName`, `Estimate`, `StdError`, `Zvalue`, `Pvalue`, and `restrict`.
- `ef`: Observation-level efficiency results (unrestricted and restricted), including the ID variables in `identifiers`, weights, efficiency measures (e.g., `u`), model-fitted values (`mlFitted`), and `restrict`.
- `el`: Observation-level elasticities and returns-to-scale-type measures. Includes IDs, weights, elasticity columns (e.g., `el1`, `el2`, ..., and the summed elasticity), and `restrict`.
- `rk`: Observation-level risk measures (unrestricted and restricted), including IDs, weights, and `risk`, plus `restrict`.

See Also

Other frontier analysis: `msf_workhorse()`

`study_dir` *Directory accessors*

Description

Resolve a study's figure, figure-data or table directory. Package code and study scripts should use these rather than pasting `"figure"` or `"figure_data"` next to `wd$output`.

Usage

```
study_dir_figures(study_environment)

study_dir_figure_data(study_environment)

study_dir_tables(study_environment)
```

Arguments

`study_environment`
: A study environment from `study_setup()` or `study_dirs()`.

Details

Under the `v2` layout `study_dir_figure_data()` and `study_dir_figures()` return the same path – plots and their underlying data share a folder.

Value

Length-1 character path. The directory is created if absent.

See Also

`study_dirs()`

study_dirs *Repair and create a study's directories*

Description

Backfills study_environment$wd from project_name and creates every directory it names. Returns the repaired environment. This is the single definition of the directory tree; study_setup() is a thin wrapper over it.

Usage

```
study_dirs(
  study_environment = NULL,
  project_name = NULL,
  layout = NULL,
  create = TRUE
)
```

Arguments

study_environment	A study environment from study_setup(), or NULL to build a fresh wd from project_name alone.
project_name	Length-1 character. Defaults to study_environment$project_name, then to the basename inferred from wd$home.
layout	Optional layout override. See .study_layouts(). Defaults to study_environment$layout, then "legacy".
create	Create the directories? Default TRUE.

Value

study_environment with wd, project_name and layout repaired.

Why this exists

wd is written into <project>_study_environment.rds by study_setup(), which only 000/001 call, so it is a *frozen snapshot*: a directory added to the layout does not reach later stages until the environment is regenerated – for most studies, an expensive re-run of matching. A stage reading an older .rds gets NULL for a new entry, and file.path(NULL, "x.rds") returns character(0) rather than erroring, which surfaces far from its cause as a gzfile() connection failure.

Calling study_dirs() after readRDS() makes wd advisory: paths are recomputed from project_name, missing entries filled, folders created.

See Also

study_setup(), study_dir_figures(), study_dir_tables()

Examples

```
## Not run:
se <- readRDS("studies/land_tenure/data/land_tenure_study_environment.rds")
se <- study_dirs(se, layout = "v2")
ggplot2::ggsave(file.path(study_dir_figures(se), "trend.png"), fig)

## End(Not run)
```

`study_setup` *Initialize Study Environment and Directory Structure*

Description

Creates the study's directory tree and returns the environment object every later stage reads from `<project>_study_environment.rds`.

Usage

```
study_setup(myseed = 1980632, project_name, layout = c("legacy", "v2"))
```

Arguments

`myseed`	Numeric/integer scalar seed. `NULL` takes `okwaayeli_control()$myseed`. Default: 1980632.
`project_name`	Length-1, non-NA character project name (required).
`layout`	Output layout: `"legacy"` (`figure/` + `figure_data/`) or `"v2"` (`figures/` holding plots and their data, plus `tables/`). See `.study_layouts()`. Defaults to `"legacy"`; land_tenure passes `"v2"`.

Details

A thin wrapper over `study_dirs()` plus the seed. The tree itself is defined once, in `.study_layouts()`.

Value

List with `wd` (directories), `project_name`, `layout` and `myseed`.

Directories

`wd` travels inside the saved `.rds` and is therefore a snapshot of whatever this function produced on the run that wrote it. Stages that `readRDS()` the environment should call `study_dirs()` on it to recompute the paths and create the folders – see the note in `study_dirs()` for why treating `wd` as authoritative caused a silent failure on 2026-07-16.

Prefer `study_dir_figures()`, `study_dir_figure_data()` and `study_dir_tables()` over reaching into `wd` directly.

See Also

`study_dirs()`, `study_dir_figures()`, `okwaayeli_control()`

tab_main_specification *Collect Main-Specification Frontier Results*

Description

Reads a set of estimation `.rds` files and binds their summaries into one `data.frame` for the figure builders and the main-specification table.

Usage

```
tab_main_specification(res_list, study_environment)
```

Arguments

`res_list`	Character vector of paths to estimation `.rds` files, normally built from `study_environment$wd$esti`
`study_environment`	The study environment list from `study_setup()` / `001_DATA_*.R`.

Details

Each file is read inside a `tryCatch`, so an unreadable or half-written estimation is skipped rather than aborting the run. If a figure comes back empty, check that every path in `res_list` exists before suspecting the plot.

Value

A `data.frame` of stacked results.

See Also

Other exhibit figures: `ers_theme()`, `fig_covariate_balance()`, `fig_distribution()`, `fig_dsistribution()`, `fig_heterogeneity00()`, `fig_input_te()`, `fig_robustness()`

treatment_effect_calculation *Compute log-linear treatment effects (ATE/ATET/ATEU) for multiple outcomes*

Description

For a given matching specification index `i`, this function:

1. Loads the corresponding matched weights file from `matching_output_directory`,
2. Merges those weights into the analysis `data` via `unique_identifier`,
3. Optionally normalizes each outcome by the first element of `outcome_variables` (e.g., to construct per-hectare or per-unit measures),
4. Fits a weighted log-linear model for each outcome with interactions between `Treat` and the supplied matching covariate formulas,
5. Transforms fitted differences into percentage treatment effects, trims extreme values, and computes weighted ATE, ATET, and ATEU alongside model fit statistics.

Usage

```
treatment_effect_calculation(
  data,
  outcome_variables,
  normalize = NULL,
  i,
  match_specifications,
  matching_output_directory,
  match_formulas
)
```

Arguments

data	A data.frame containing at least: • unique_identifier: unit identifier used for merging matched weights, • Treat: treatment indicator (logical or 0/1), • Area: plot size (positive; used for input checks and typical scaling), • all variables named in outcome_variables, • any covariates referenced in match_formulas$general_match and match_formulas$exact_ma
outcome_variables	Character vector of outcome variable names (e.g., c("HrvstKg","Area","SeedKg","HHLaborAE",""
normalize	Logical scalar. If TRUE, each outcome in outcome_variables is divided by the first element of outcome_variables in the merged data (e.g., to obtain outcomes per hectare or per unit).
i	Integer index selecting the row of match_specifications and the corresponding matching result file (zero-padded via ARRAY to "NNNN.rds").
match_specifications	A data.frame with at least a column ARRAY. The i-th row is used to locate the matching output file and is cbind-ed to the returned results.
matching_output_directory	Directory containing per-specification matching results named "match_0001.rds", "match_0002.rds", etc. Each RDS is expected to contain an element $md with columns unique_identifier and weights.
match_formulas	A list with: • general_match: a formula; the RHS is accessed via as.character(...)[3], • exact_match: a formula; the RHS is accessed via as.character(...)[2].

Details

For each outcome oc in outcome_variables, the function:

1. Filters to observations with non-missing Treat, the outcome, and weights,
2. Constructs a log-linear model of the form

   log(oc + eps) ~ Treat * (general_match_rhs + exact_match_rhs)

   where eps = min(oc[oc > 0]) * 1/100 in the estimation sample,
3. Fits the model using lm() with weights given by the matched weights,
4. Predicts fitted log outcomes for treated (Treat = 1) and untreated (Treat = 0) counterfactuals,

5. Computes individual percentage treatment effects

$$TE_OLS = \{\exp(\hat{y}_1 - \hat{y}_0) - 1\} \times 100,$$

trims them to the interval $[-100, 100]$, and then forms:

- ATE: weighted mean of `TE_OLS` over all units,
- ATET: weighted mean of `TE_OLS` among treated units,
- ATEU: weighted mean of `TE_OLS` among untreated units.

In addition, for each outcome the function records model diagnostics: AIC, log-likelihood, R^2, adjusted R^2, F-statistic, and the sample size used in the regression.

Value

A `data.frame` (`atet_scalar`) with the columns from `match_specifications[i, ]` repeated across rows, and the additional columns:

- `outcome`: outcome variable name,
- `treatment`: name of the treatment indicator (always `"Treat"`),
- `level`: one of `"ATE"`, `"ATET"`, `"ATEU"`, `"aic"`, `"ll"`, `"R2"`, `"N"`, `"Ft"`, `"R2a"`,
- `est`: numeric estimate corresponding to `level`.

One block of rows is returned per outcome in `outcome_variables`. If a model fails for a given outcome, that outcome is skipped.

See Also

Other matching: `covariate_balance()`, `draw_matched_samples()`, `match_sample_specifications()`, `treatment_effect_summary()`, `write_match_formulas()`

`treatment_effect_summary` *Summarize Treatment Effect Estimates Across Matching Specifications*

Description

Aggregates and summarizes treatment effect results stored as `.rds` files in a specified output directory. The function reads each RDS file, combines all results into a single data frame, filters out invalid estimates, and computes jackknife-style summary statistics (mean, standard error, sample size, t-value, and p-value) by matching specification and outcome.

Usage

```
treatment_effect_summary(treatment_effects_output_directory)
```

Arguments

`treatment_effects_output_directory`
: Character string giving the path to the directory containing `.rds` files with treatment effect results (e.g., outputs of `treatment_effect_calculation()` saved across specifications).

Details

The function proceeds through the following steps:

1. Lists and reads all `.rds` files in `treatment_effects_output_directory`.
2. Combines them into a single long-format data frame using `data.table::rbindlist()`.
3. Removes rows with invalid estimates (`NA`, `NaN`, `Inf`, or `-Inf`).
4. Separates base (non-bootstrap) results where `boot == 0` and joins these with jackknife summary statistics computed via `doBy::summaryBy()` across method, distance, link, outcome, and level.
5. Renames summary columns to:
   - `jack_mean`: mean estimate
   - `jack_se`: standard error
   - `jack_n`: sample size (number of replicates)
   - `jack_tvl`: t-value (`jack_mean / jack_se`)
   - `jack_pvl`: two-sided p-value

Value

A `data.frame` containing summarized treatment effect estimates with columns:

- `method`, `distance`, `link`, `outcome`, `level`
- `est` (original estimate for non-bootstrap sample)
- `jack_mean`, `jack_se`, `jack_n`, `jack_tvl`, `jack_pvl`

See Also

Other matching: `covariate_balance()`, `draw_matched_samples()`, `match_sample_specifications()`, `treatment_effect_calculation()`, `write_match_formulas()`

`write_match_formulas` *Construct Matching Formulas for Exact and General Matching*

Description

Builds two types of matching formulas commonly used in propensity score or covariate matching workflows:

- An **exact match** formula specifying categorical variables to match exactly.
- A **general match** formula specifying numeric and factor covariates for distance-based matching.

The function returns both formulas as character strings that can be used in matching functions such as `MatchIt::matchit()` or `Matching::Match()`.

Usage

```
write_match_formulas(
  match_variables_exact,
  match_variables_scaler,
  match_variables_factor
)
```

Arguments

`match_variables_exact`
: A character vector of variable names to be included in the exact match component (e.g., `"gender"`, `"region"`).

`match_variables_scaler`
: A character vector of continuous or scalar variable names to include as covariates in the general match formula (e.g., `"age"`, `"income"`).

`match_variables_factor`
: A character vector of factor variable names to be included as dummy-coded categorical covariates in the general match formula.

Value

A named list with two elements:

`exact_match` A string representing the exact match formula (factor variables only).

`general_match` A string representing the general match formula, including both continuous and factor covariates on the right-hand side of `Treat ~`.

See Also

Other matching: `covariate_balance()`, `draw_matched_samples()`, `match_sample_specifications()`, `treatment_effect_calculation()`, `treatment_effect_summary()`

Index